Lesson 01 Lesson-End Project

Deploying EC2 Instances with Terraform and Custom VPC

Project agenda: To demonstrate the process of launching an Amazon EC2 instance within a custom VPC using Terraform for automated infrastructure deployment and management

Description: Your project manager wants a secure and isolated AWS setup for critical applications. You are tasked to deploy an EC2 instance in a custom VPC using Terraform, an Infrastructure as Code tool. This will ensure applications run in a controlled environment with automated deployment. Terraform scripts will define AWS resources, ensuring standardization and error reduction. Once executed, instances will be active within the VPC.

Tools required: AWS Management Console

Prerequisites: A custom VPC with a public subnet and an active EC2 instance

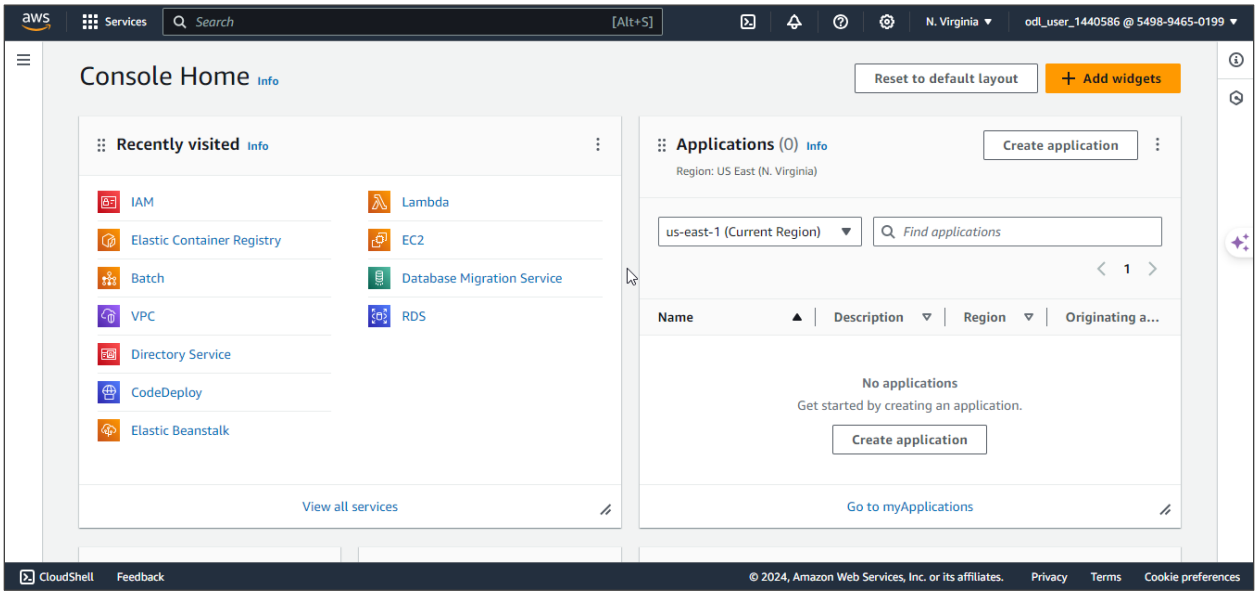
Expected deliverables: A functioning CI/CD pipeline on AWS for automated deployments

Steps to be followed:

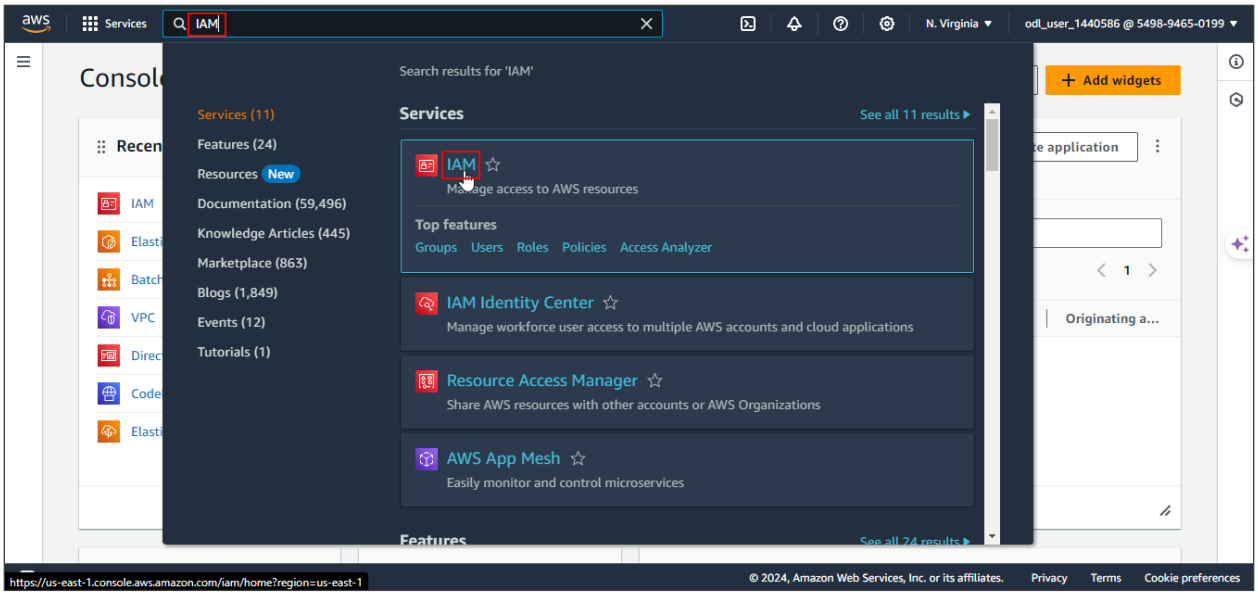
1. Create an IAM access key
2. Create a Terraform to launch the VM

Step 1: Create an IAM access key

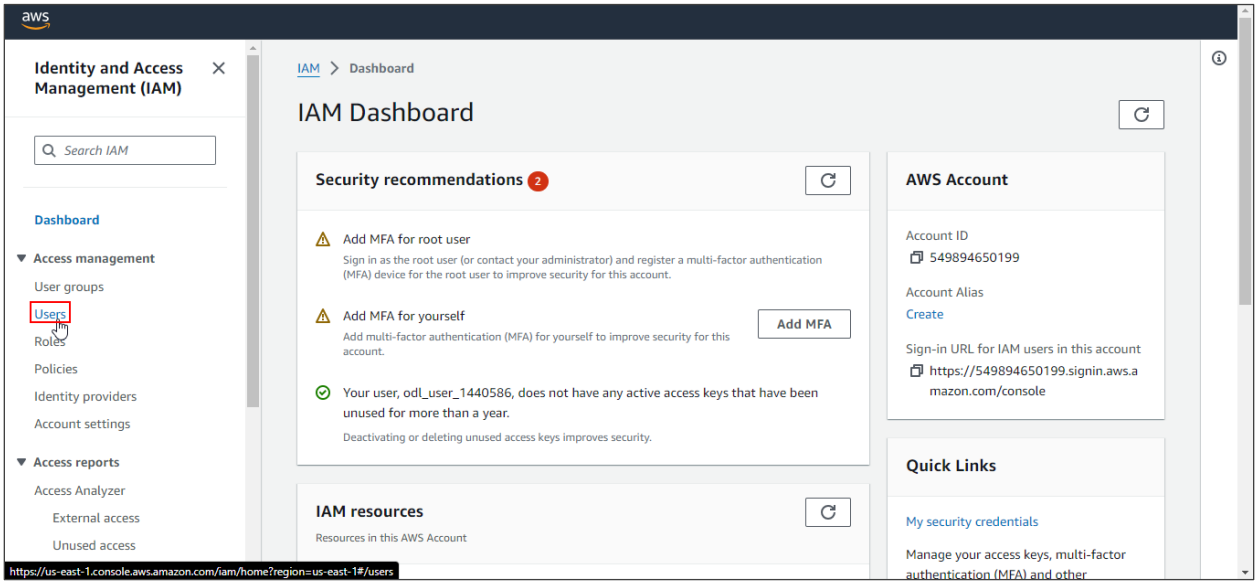
1.1 Navigate to the AWS Management Console



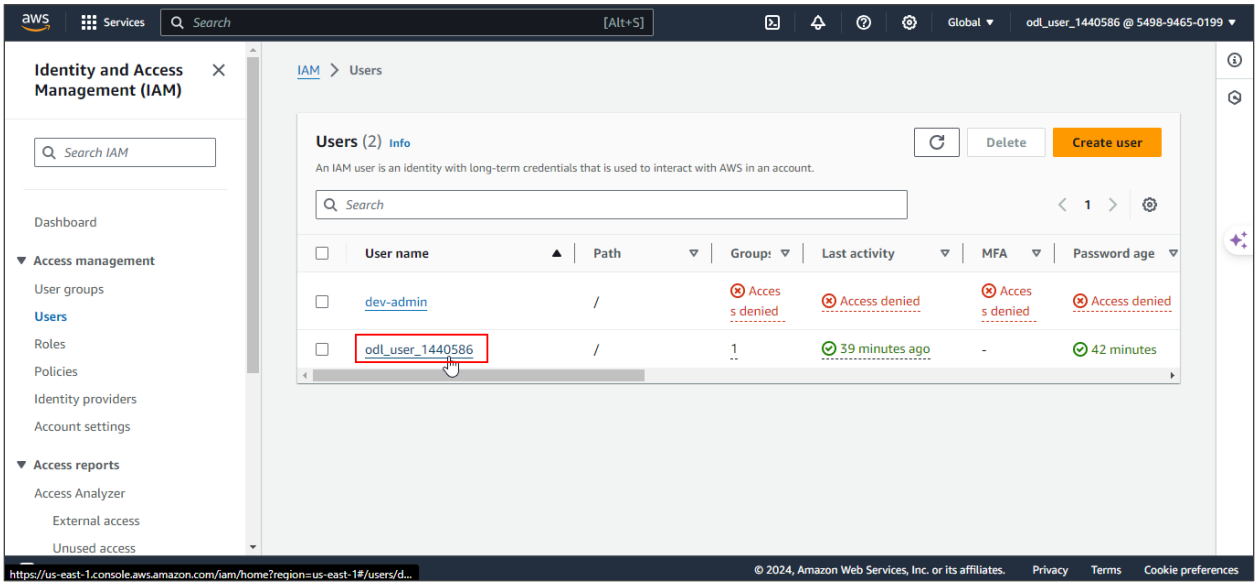
1.2 Search for and click on **IAM**



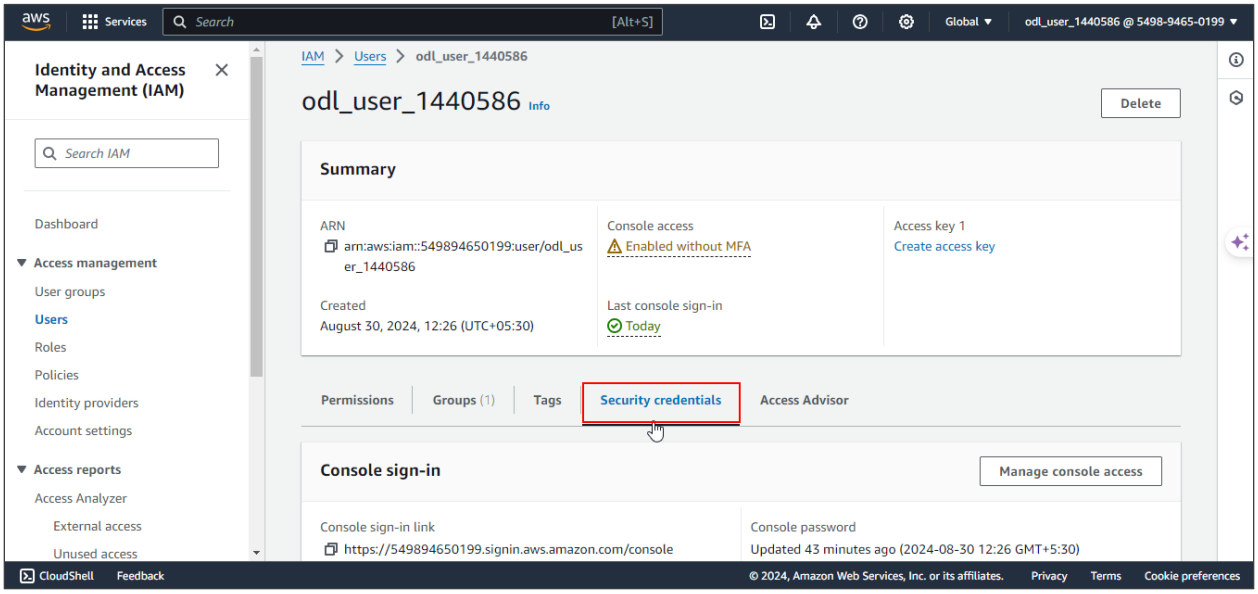
1.3 Click on **Users** under the **Access management** section



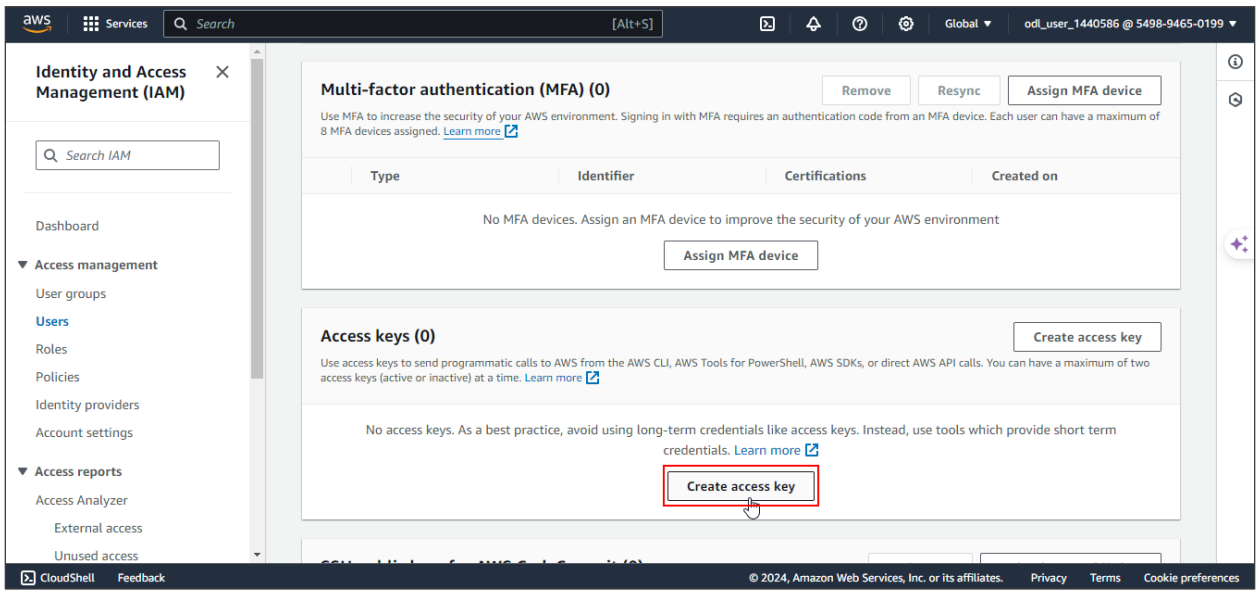
1.4 Click on the IAM user name



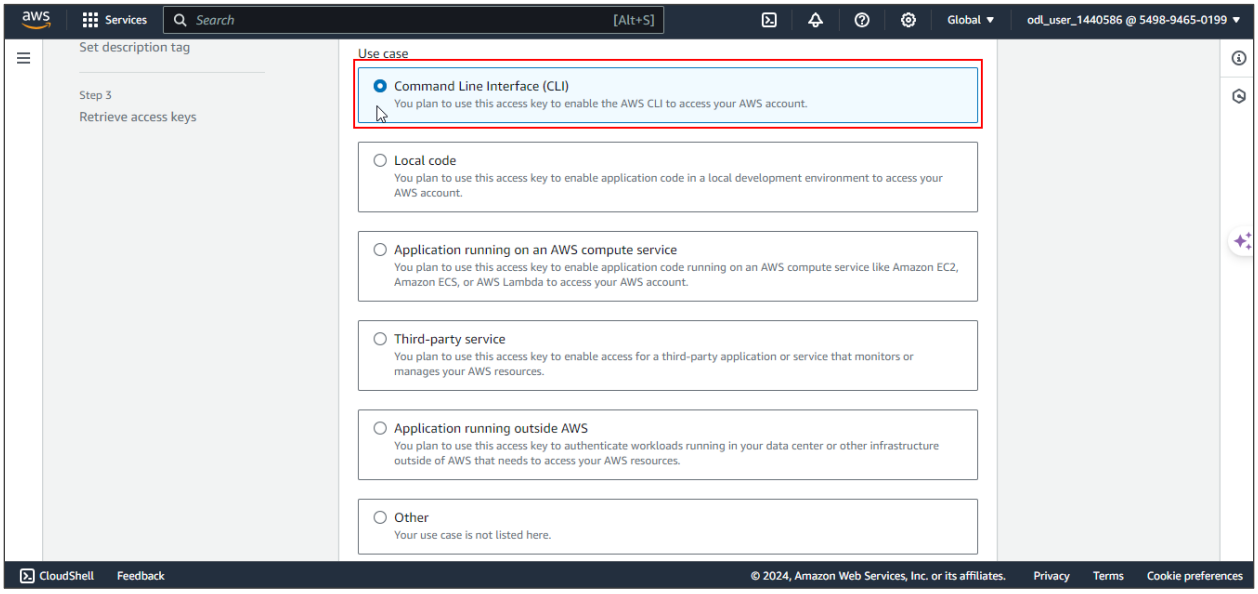
1.5 Select **Security credentials**



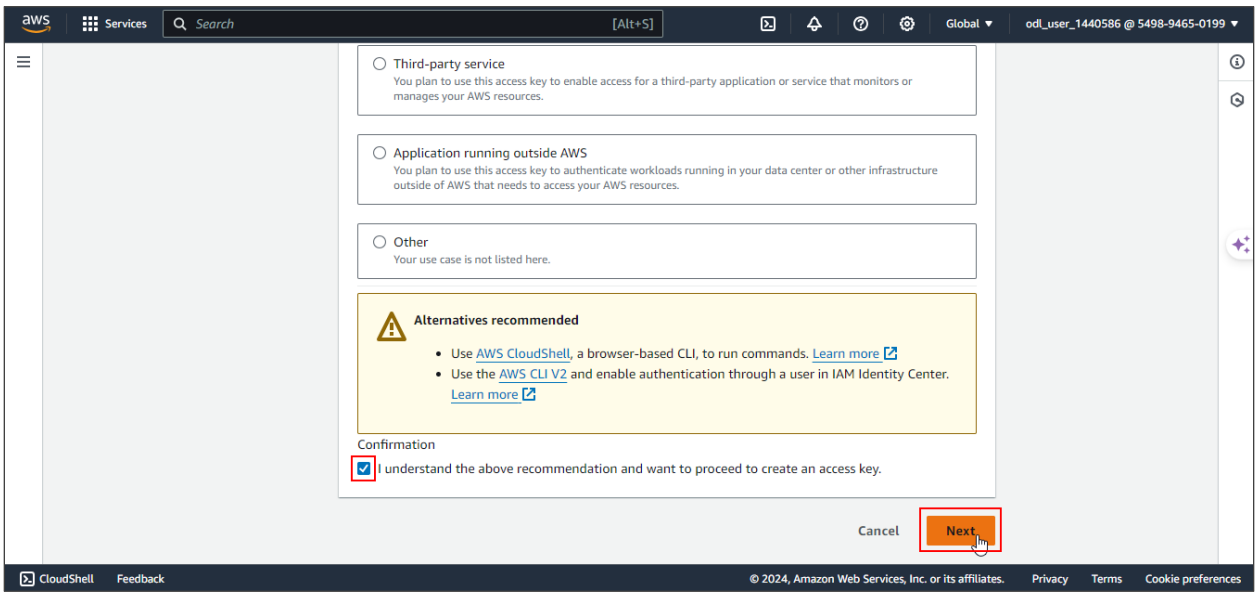
1.6 Click **Create access key**



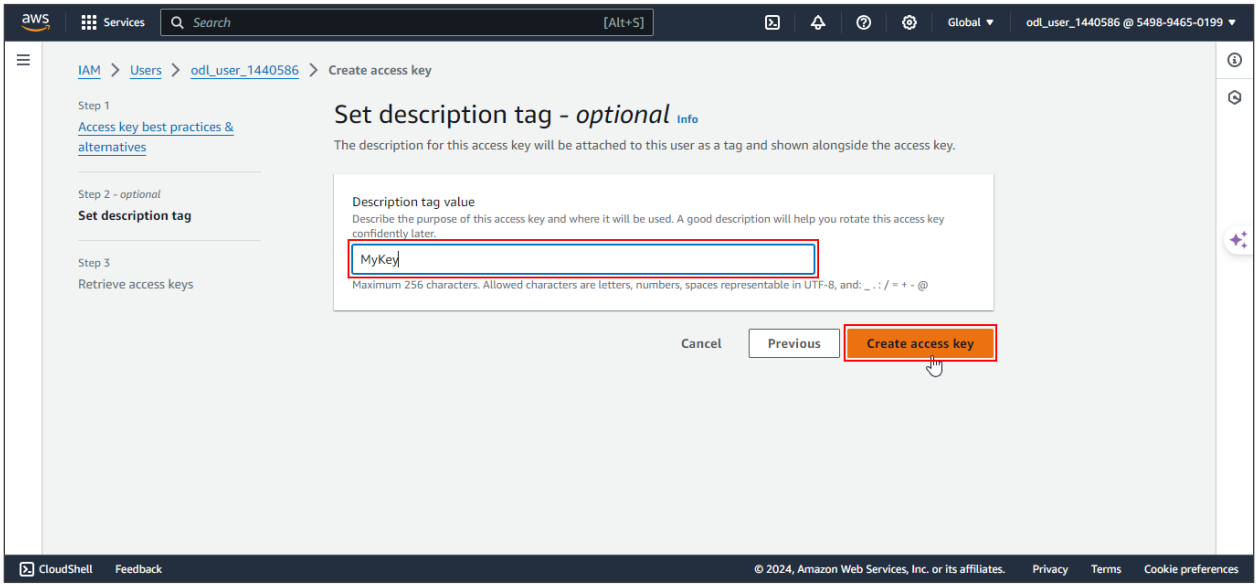
1.5 Choose **Command Line Interface (CLI)**



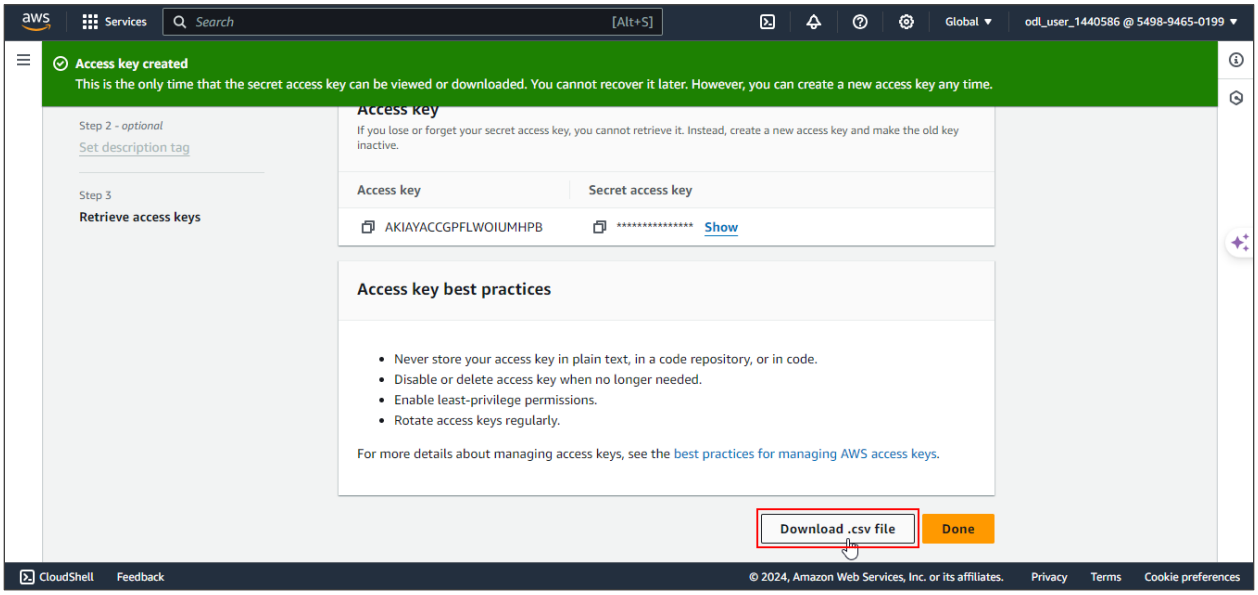
1.6 Click on the check box and then click **Next**



1.7 Enter a desired description and click on **Create access key**

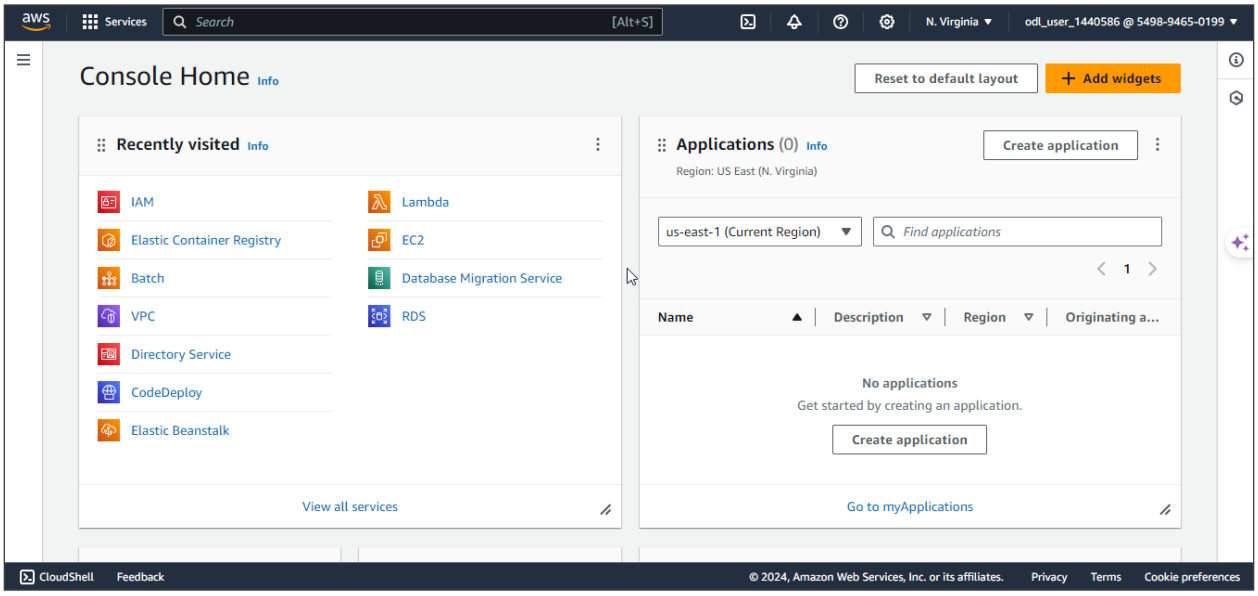


1.8 Click **Download .csv file**

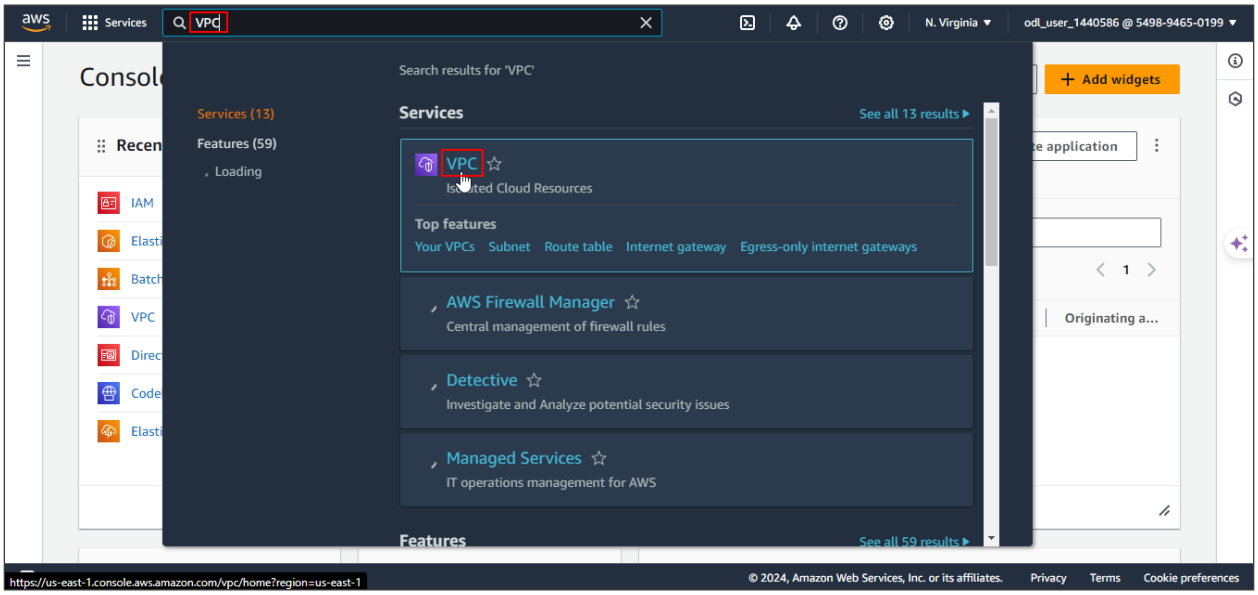


Step 2: Create a Terraform to launch the VM

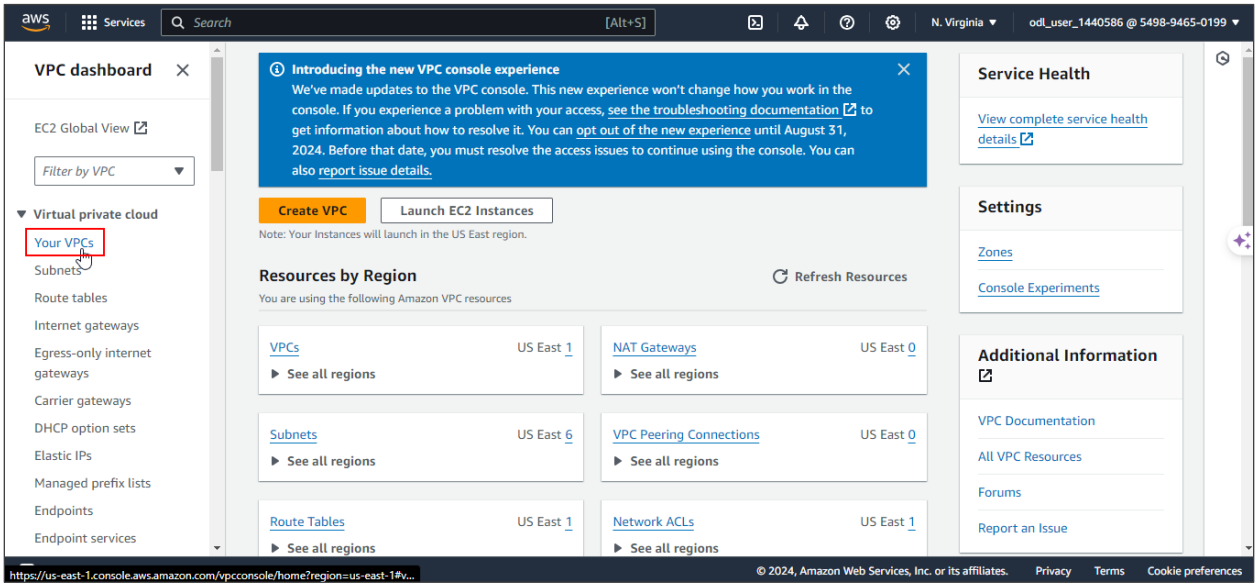
2.1 Navigate to the AWS Management Console



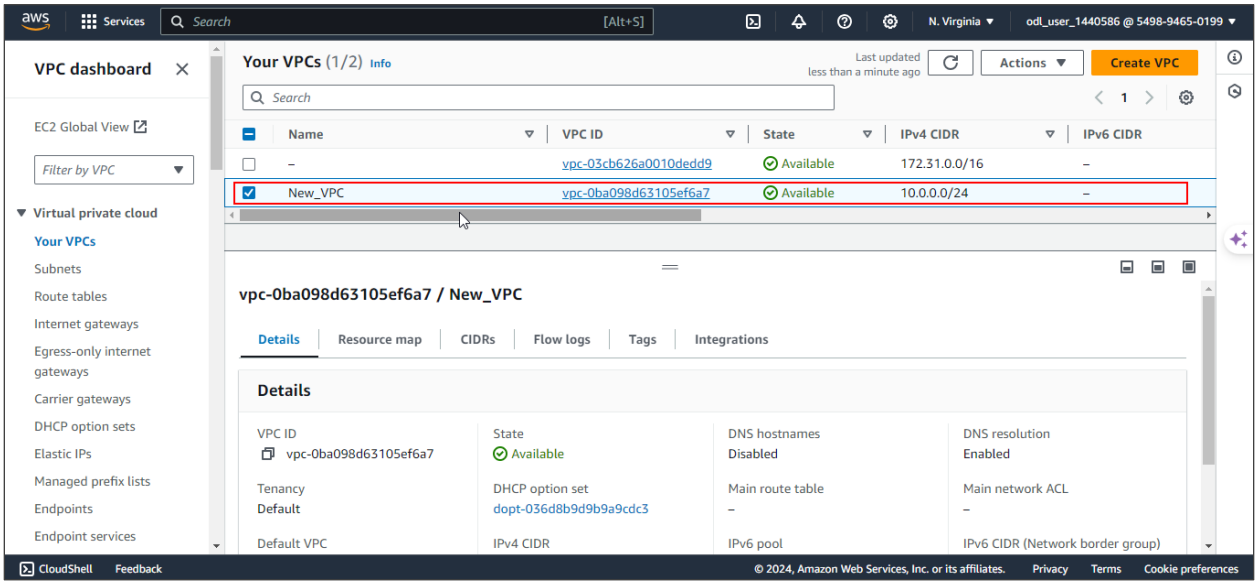
2.2 Search for and click on **VPC**



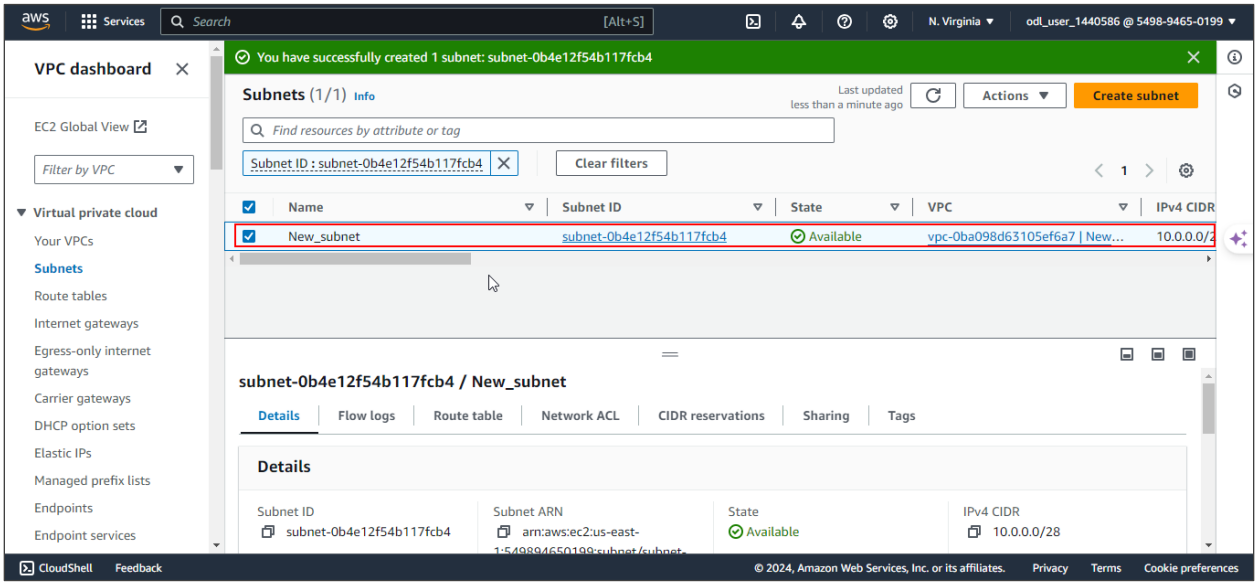
2.3 Click on **Your VPCs** under the **Virtual private cloud** section



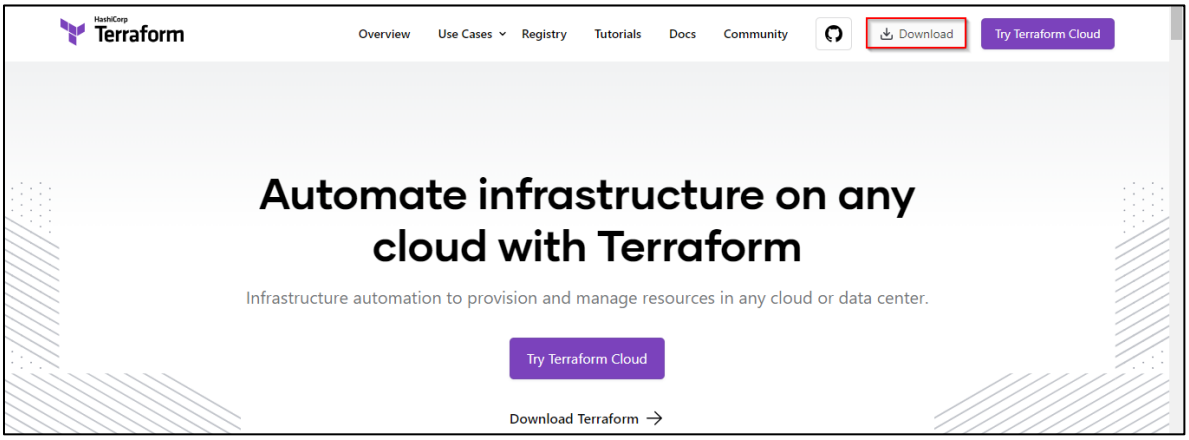
2.4 Create a custom VPC named **New_VPC**



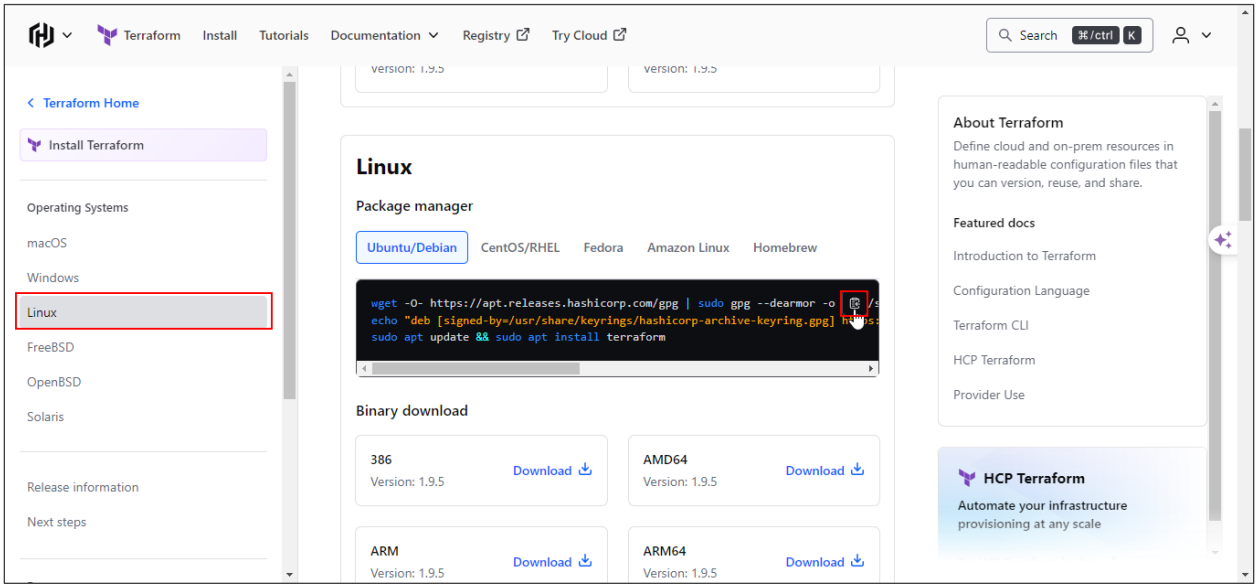
2.5 Create a public subnet within the newly created VPC



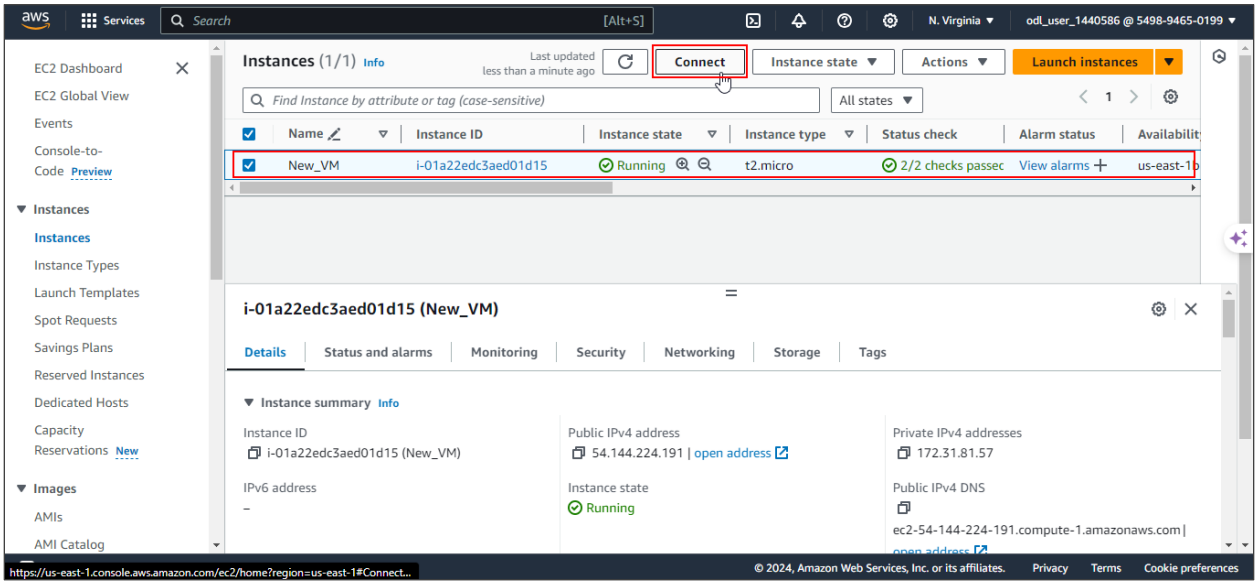
2.6 Navigate to the following link and click on **Download** to download Terraform: https://www.terraform.io/



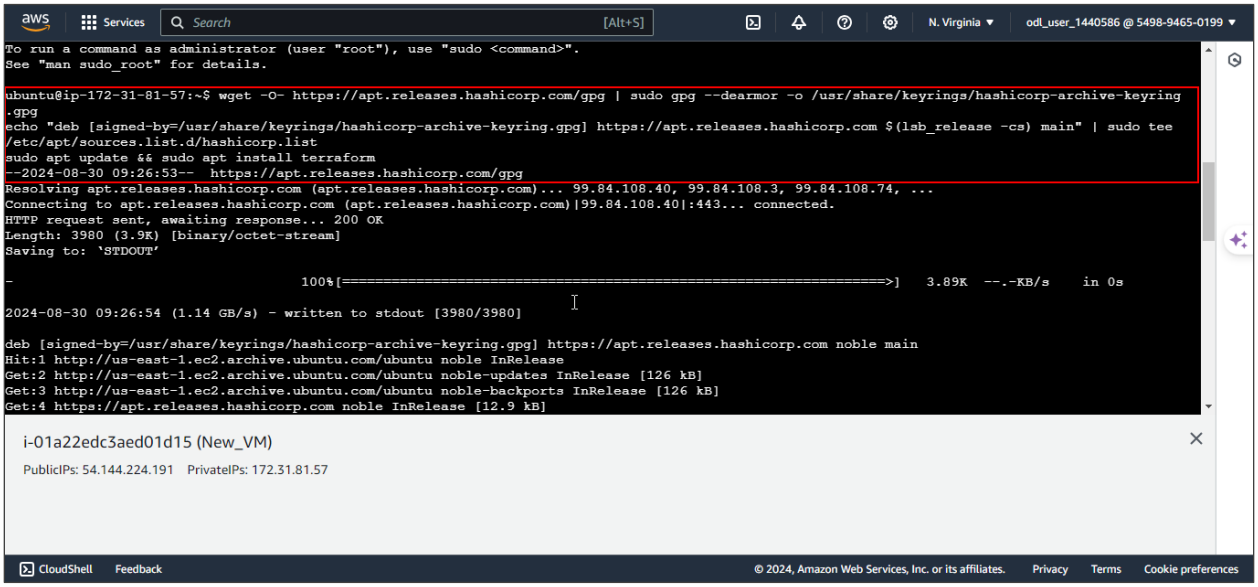
2.7 Select **Linux** and copy the **Ubuntu/Debian** commands



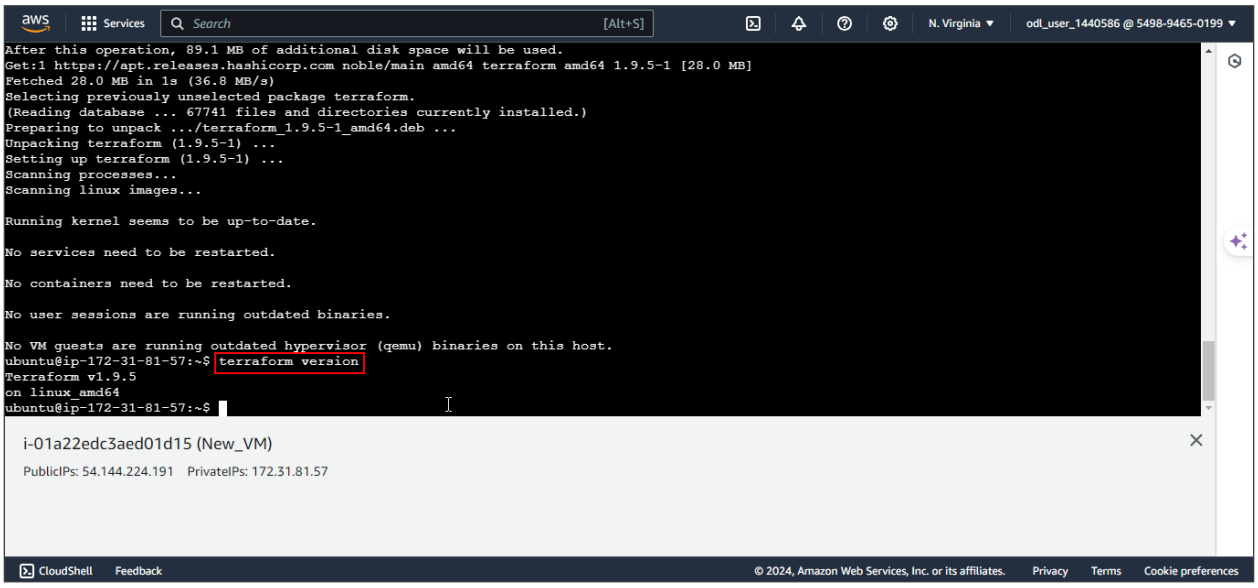
2.8 Create an EC2 Instance and click on the **Connect** button



2.9 Paste the copied code into the EC2 instance terminal



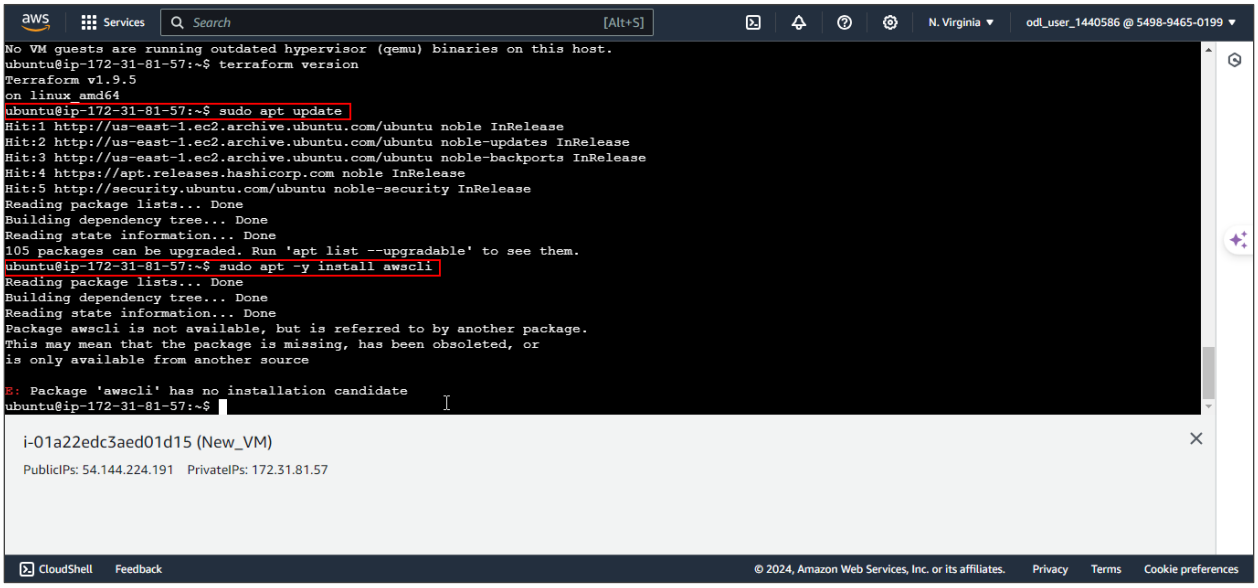
2.10 Enter the following command to check the Terraform version:
terraform version



2.11 Enter the following commands to set up the AWS CLI:

sudo apt update

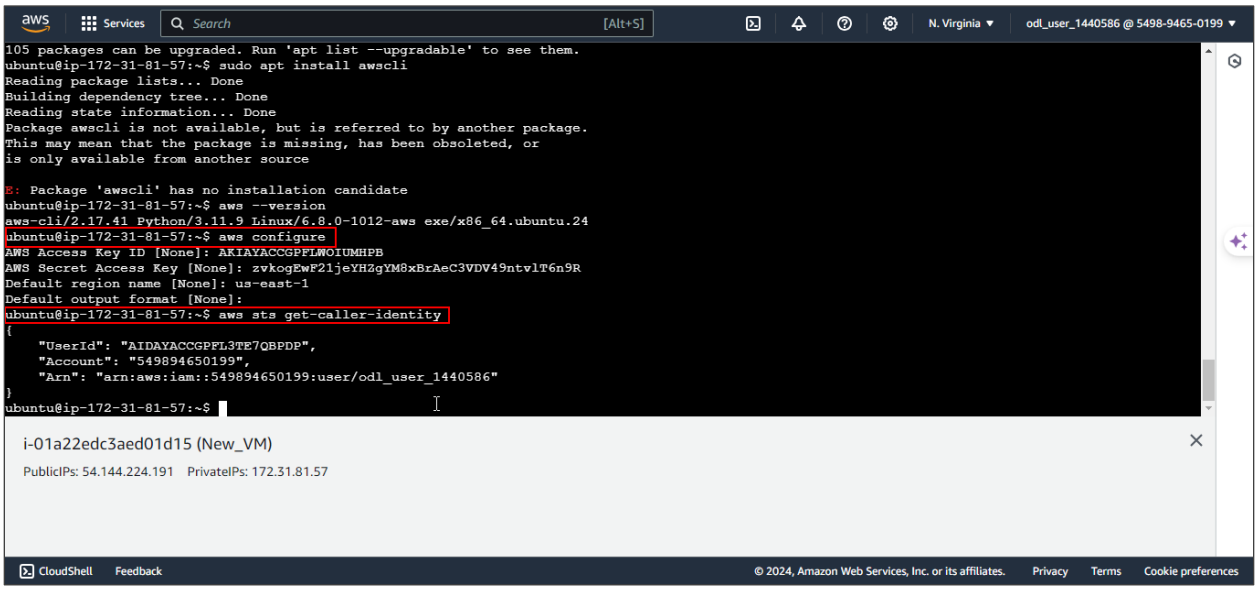
sudo apt -y install awscli



2.12 Use the following commands to configure the AWS Access Key ID, Default region name, and Default output format:

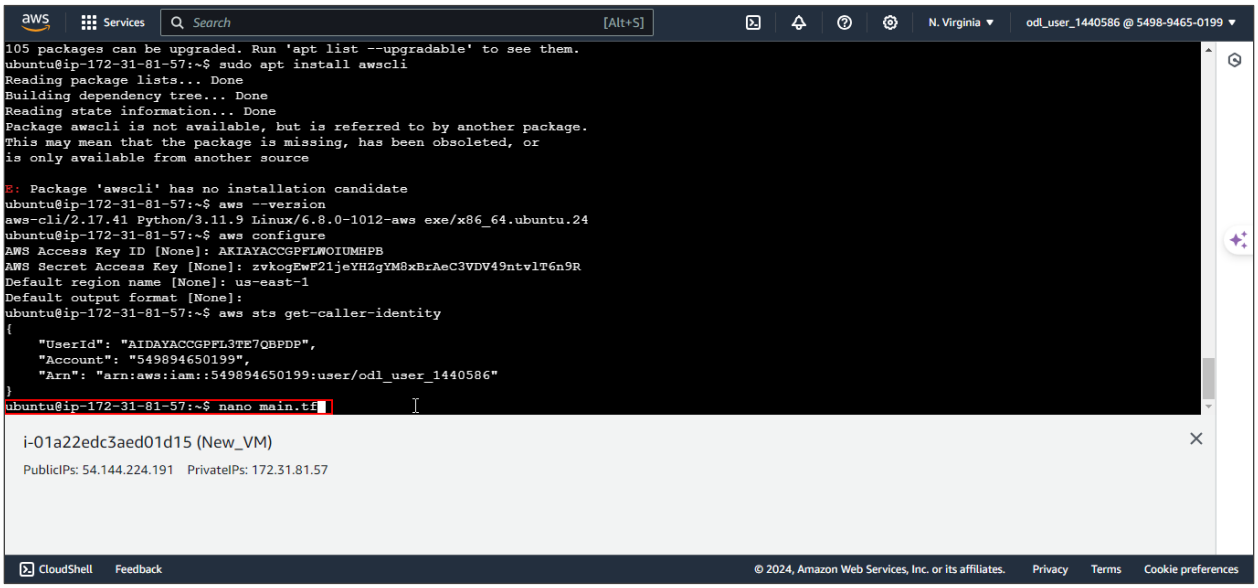
aws configure

aws sts get-caller-identity



2.13 Enter the following command to edit the **main.tf** file:

nano main.tf



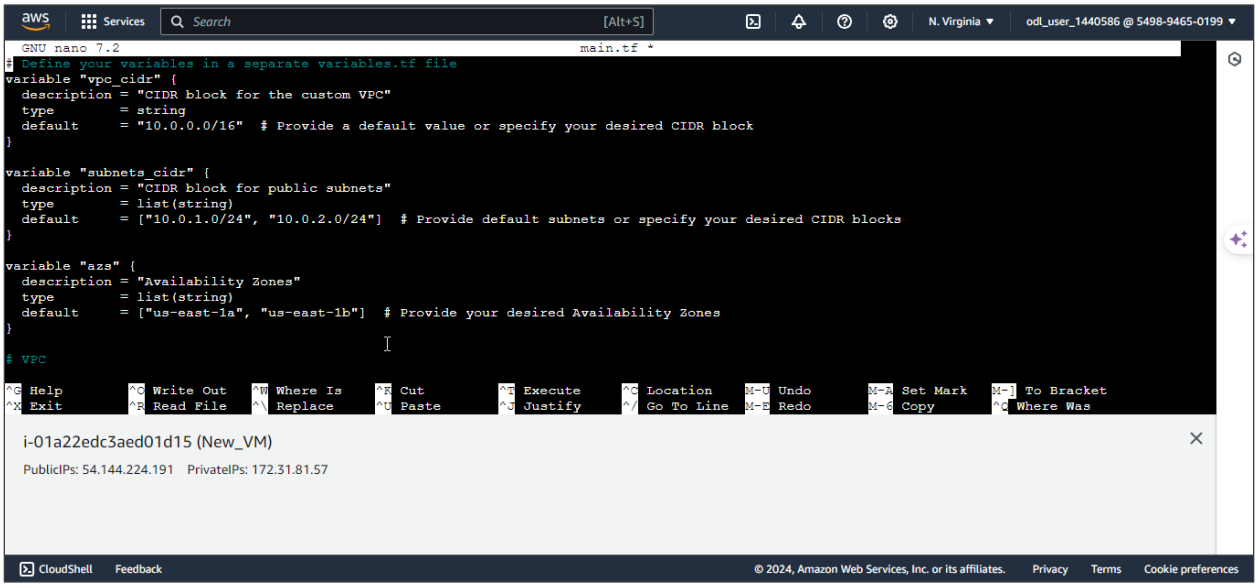
2.14 Enter the code below into the text editor (nano):

```
# Define your variables in a separate variables.tf file
variable "vpc_cidr" {
  description = "CIDR block for the custom VPC"
  type        = string
  default     = "10.0.0.0/16"  # Provide a default value or specify your desired CIDR
block
}

variable "subnets_cidr" {
  description = "CIDR block for public subnets"
  type        = list(string)
  default     = ["10.0.1.0/24", "10.0.2.0/24"]  # Provide default subnets or specify your
desired CIDR blocks
}

variable "azs" {
  description = "Availability Zones"
  type        = list(string)
  default     = ["us-east-1a", "us-east-1b"]  # Provide your desired Availability Zones
}
```

```
# VPC
resource "aws_vpc" "terra_vpc" {
  cidr_block = var.vpc_cidr
  tags = {
    Name = "TerraVPC"
  }
}

# Internet Gateway
resource "aws_internet_gateway" "terra_igw" {
  vpc_id = aws_vpc.terra_vpc.id
  tags = {
    Name = "main"
  }
}

# Subnets: public
resource "aws_subnet" "public" {
  count          = length(var.subnets_cidr)
  vpc_id         = aws_vpc.terra_vpc.id

  cidr_block      = var.subnets_cidr[count.index]

  availability_zone = var.azs[count.index]
  map_public_ip_on_launch = true
  tags = {
    Name = "Subnet-${count.index + 1}"
  }
}

# Route table: attach Internet Gateway
resource "aws_route_table" "public_rt" {
  vpc_id = aws_vpc.terra_vpc.id
}

# Route table association with public subnets
resource "aws_route_table_association" "a" {
  count          = length(aws_subnet.public)
  subnet_id      = aws_subnet.public[count.index].id
  route_table_id = aws_route_table.public_rt.id
}
```

```
resource "aws_security_group" "jenkins_security_group" {
  name        = "sg_jenkins"
  description = "jenkins security group."
  vpc_id      = aws_vpc.terra_vpc.id
}

resource "aws_security_group_rule" "ssh_ingress_access" {
  type        = "ingress"
  from_port   = 22
  to_port     = 22
  protocol    = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
  security_group_id = aws_security_group.jenkins_security_group.id
}

resource "aws_security_group_rule" "egress_access" {
  type        = "egress"
  from_port   = 0
  to_port     = 65535
  protocol    = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
  security_group_id = aws_security_group.jenkins_security_group.id

}

data "aws_ami" "latest-ubuntu" {

  most_recent = true

  filter {
    name   = "name"
    values = ["amzn2-ami-kernel-5.10-hvm-2.0.20230418.0-x86_64-gp2"]
  }

  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }
}
```

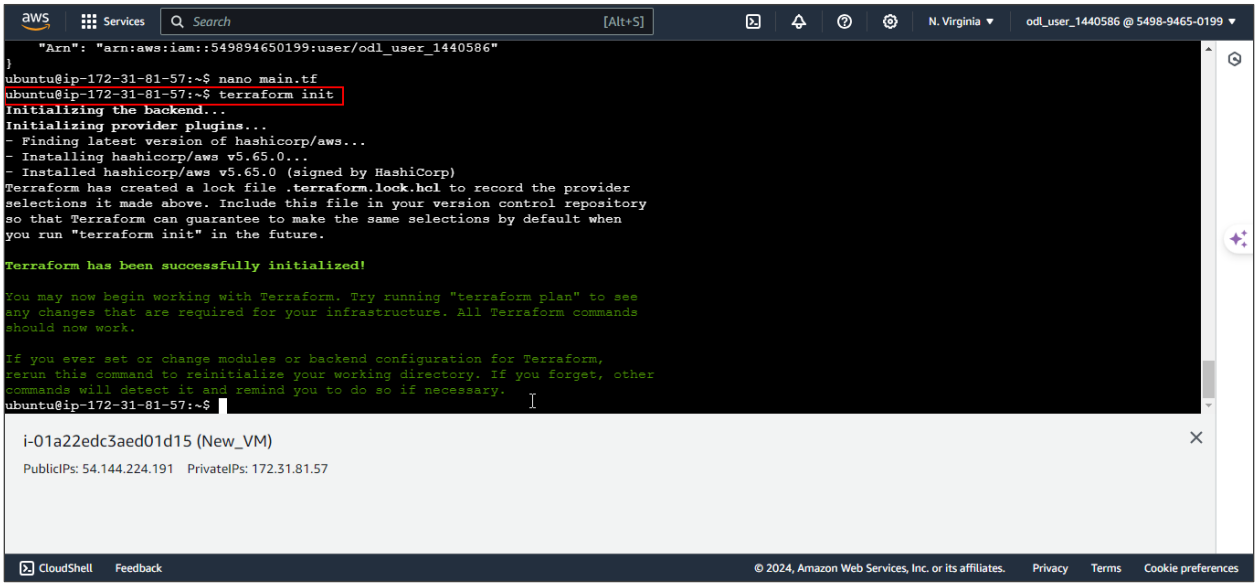
```
resource "aws_instance" "jenkins_instance" {
  instance_type          = "t2.micro"
  vpc_security_group_ids    = [aws_security_group.jenkins_security_group.id]
  associate_public_ip_address = true
  tags = {
    Name = "jenkins-instance"
  }
  ami          = data.aws_ami.latest-ubuntu.id
  availability_zone = var.azs[0]  # Choose the first Availability Zone from your list
  subnet_id     = aws_subnet.public[0].id  # Choose the first public subnet from your list
}

# Optionally, you can create additional instances in other subnets
resource "aws_instance" "jenkins_instance2" {
  instance_type          = "t2.micro"
  vpc_security_group_ids    = [aws_security_group.jenkins_security_group.id]
  associate_public_ip_address = true
  tags = {
    Name = "jenkins-instance2"
  }
  ami          = data.aws_ami.latest-ubuntu.id
  availability_zone = var.azs[1]  # Choose the second Availability Zone from your list
  subnet_id     = aws_subnet.public[1].id  # Choose the second public subnet from your
list
}
```

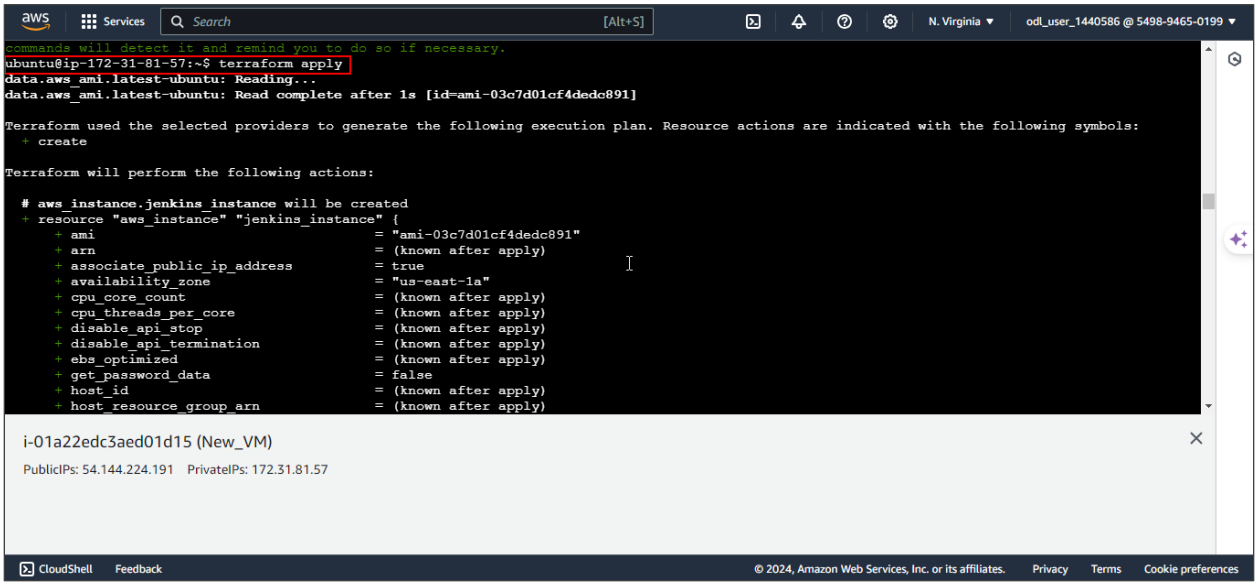


2.15 Enter the following commands to apply the terraform:

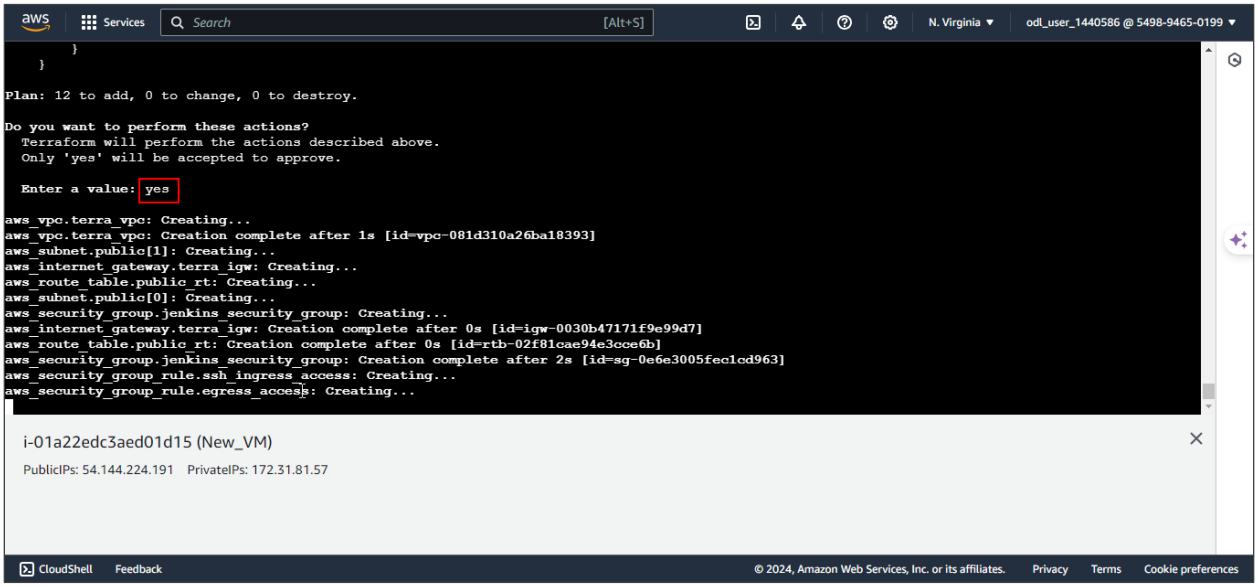
terraform init

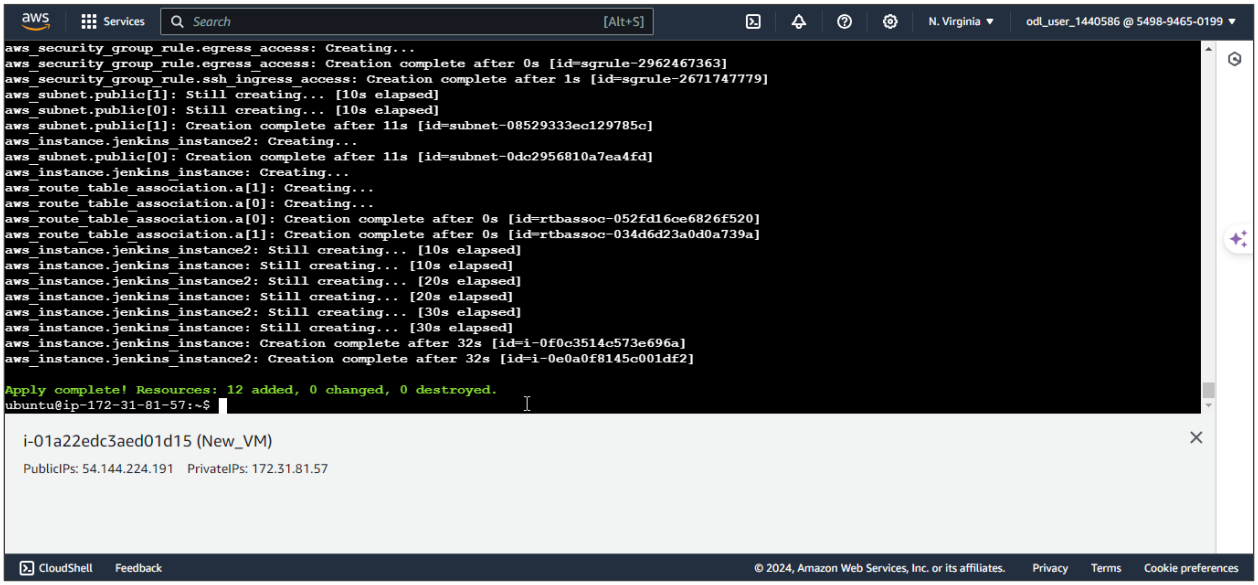
terraform apply



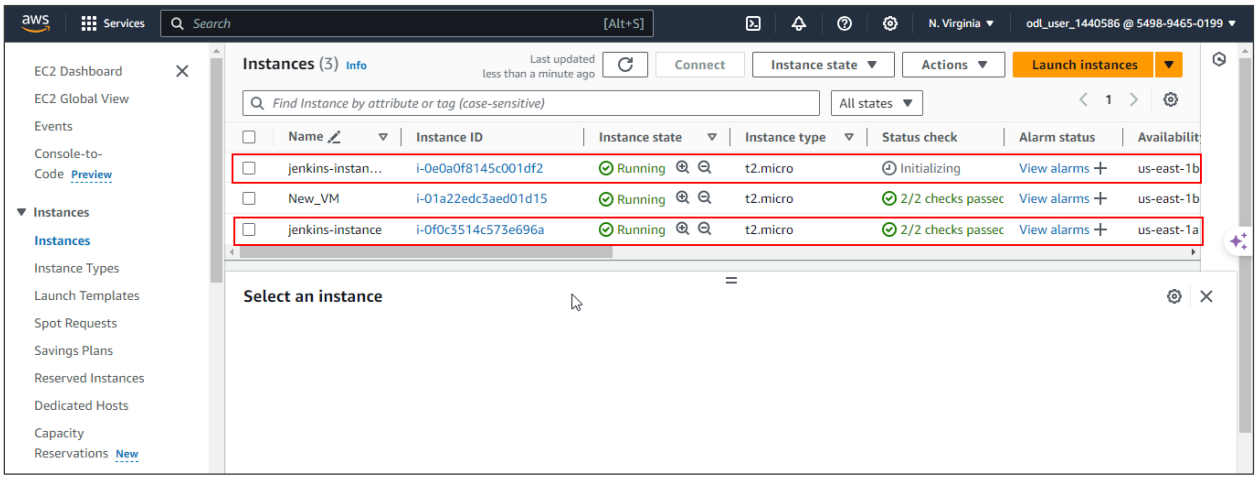


2.16 Enter **yes** to confirm





2.17 Navigate to the **Instances** section to validate the availability of the new VM within the custom VPC



By following these steps, you have successfully launched an Amazon EC2 instance within a custom VPC using Infrastructure as Code with Terraform for AWS.